

PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

PURPLLE ONLINE BEAUTY SHOPPING

1. Introduction

The **Purplle Online Beauty Shopping Database Management System** is designed to manage product reviews and ratings provided by customers. This week focuses on creating and managing the **Rating** table. The system stores customer ratings for products and uses SQL queries and aggregate functions to calculate average ratings and identify highly rated products.

2. Objectives

The main objectives of this week are:

- To create the **Rating** table.
- To store customer ratings for products.
- To retrieve product rating details.
- To calculate average product ratings using aggregate functions.
- To identify highly rated products.
- To generate product rating reports.

3. Database design

3.1 Review Table

Attribute	Data Type	Description
Review_ID	NUMBER	Unique ID for each review. Primary Key
Customer_ID	NUMBER	Identifies the customer who gives the review. Foreign Key
Product_ID	NUMBER	Identifies the product being reviewed. Foreign Key
Review_Text	VARCHAR2(500)	Stores the customer's written feedback about the product.
Review_Date	DATE	Stores the date on which the review was submitted.

3.2 Rating Table

Attribute	Data Type	Description
Rating_ID	NUMBER	Unique ID for each rating. Primary Key
Customer_ID	NUMBER	Identifies the customer who gives the rating. Foreign Key
Product_ID	NUMBER	Identifies the product being rated. Foreign Key
Rating	NUMBER(2,1)	Stores the product rating from 1 to 5 .
Rating_Date	DATE	Stores the date on which the rating was given.

4. Table Relationships

Customer → Rating

CUSTOMER (1) ————— (M) RATING

Customer_ID (PK) → Customer_ID (FK)

One customer can give ratings for multiple products.

PurplleProduct → Rating

PURPLLEPRODUCT (1) ————— (M) RATING

Product_ID (PK) → Product_ID (FK)

One product can receive ratings from multiple customers.

5. Table Creation

5.1 Create Review Table

```
SQL> CREATE TABLE Review (  
2     Review_ID NUMBER PRIMARY KEY,  
3     Customer_ID NUMBER NOT NULL,  
4     Product_ID NUMBER NOT NULL,  
5     Review_Text VARCHAR2(500),  
6     Rating NUMBER(2,1) CHECK (Rating BETWEEN 1 AND 5),  
7     Review_Date DATE NOT NULL,  
8     FOREIGN KEY (Customer_ID) REFERENCES Customer5(Customer_ID),  
9     FOREIGN KEY (Product_ID) REFERENCES PurplleProduct(Product_ID)  
10  );  
  
Table created.
```

5.2 Create Rating Table

```
SQL> CREATE TABLE Rating (  
2     Rating_ID NUMBER PRIMARY KEY,  
3     Customer_ID NUMBER NOT NULL,  
4     Product_ID NUMBER NOT NULL,  
5     Rating NUMBER(2,1) CHECK (Rating BETWEEN 1 AND 5),  
6     Rating_Date DATE NOT NULL,  
7     FOREIGN KEY (Customer_ID) REFERENCES Customer5(Customer_ID),  
8     FOREIGN KEY (Product_ID) REFERENCES PurplleProduct(Product_ID)  
9  );  
  
Table created.
```

6. Review Data Insertion

```
SQL> INSERT INTO Review VALUES (801, 101, 101, 'Good foundation with smooth finish', 4.5, TO_DATE('25-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Review VALUES (802, 102, 102, 'Nice lipstick and long lasting', 4.0, TO_DATE('26-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Review VALUES (803, 103, 103, 'Very effective face wash', 4.5, TO_DATE('27-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Review VALUES (804, 104, 104, 'Good serum for daily use', 5.0, TO_DATE('28-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Review VALUES (805, 105, 105, 'Good quality shampoo', 4.0, TO_DATE('29-09-2026','DD-MM-YYYY'));
1 row created.

SQL> COMMIT;
```

7. Rating Data Insertion

```
SQL> INSERT INTO Rating VALUES (901, 101, 101, 4.5, TO_DATE('25-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Rating VALUES (902, 102, 102, 4.0, TO_DATE('26-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Rating VALUES (903, 103, 103, 4.5, TO_DATE('27-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Rating VALUES (904, 104, 104, 5.0, TO_DATE('28-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Rating VALUES (905, 105, 105, 4.0, TO_DATE('29-09-2026','DD-MM-YYYY'));
1 row created.

SQL> COMMIT;

Commit complete.
```

8. Data Retrieval

8.1 Display Review Records

```
SQL> SELECT * FROM Review;

REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
REVIEW_TEXT
-----
RATING REVIEW_DA
-----
      801          101          101
Good foundation with smooth finish
      4.5 25-SEP-26

      802          102          102
Nice lipstick and long lasting
      4 26-SEP-26

REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
REVIEW_TEXT
-----
RATING REVIEW_DA
-----

      803          103          103
Very effective face wash
      4.5 27-SEP-26

      804          104          104
Good serum for daily use

REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
REVIEW_TEXT
-----
RATING REVIEW_DA
-----
      5 28-SEP-26

      805          105          105
Good quality shampoo
      4 29-SEP-26
```

8.2 Display Rating Records

```
SQL> SELECT * FROM Rating;

RATING_ID CUSTOMER_ID PRODUCT_ID    RATING RATING_DA
-----
      901          101          101      4.5 25-SEP-26
      902          102          102       4 26-SEP-26
      903          103          103      4.5 27-SEP-26
      904          104          104       5 28-SEP-26
      905          105          105       4 29-SEP-26
```

9. Retrieve Product Review Details

```
SQL> SELECT
  2     R.Review_ID,
  3     R.Customer_ID,
  4     R.Product_ID,
  5     R.Review_Text,
  6     R.Review_Date
  7 FROM Review R
  8 JOIN Customer5 C
  9 ON R.Customer_ID = C.Customer_ID
 10 ORDER BY R.Review_Date DESC;
```

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID
-----------	-------------	------------

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID
-----------	-------------	------------

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID
-----------	-------------	------------

805	105	105
Good quality shampoo		
29-SEP-26		

804	104	104
Good serum for daily use		
28-SEP-26		

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID
-----------	-------------	------------

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID
-----------	-------------	------------

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID
-----------	-------------	------------

803	103	103
Very effective face wash		
27-SEP-26		

802	102	102
Nice lipstick and long lasting		

10. Calculate Average Product Ratings

```
SQL> SELECT
  2     Product_ID,
  3     AVG(Rating) AS Average_Rating
  4 FROM Rating
  5 GROUP BY Product_ID
  6 ;
```

PRODUCT_ID	AVERAGE_RATING
------------	----------------

101	4.5
102	4
103	4.5
104	5
105	4

11. Identify Highly Rated Products

```
SQL> SELECT
  2     Product_ID,
  3     AVG(Rating) AS Average_Rating
  4 FROM Rating
  5 GROUP BY Product_ID
  6 HAVING AVG(Rating) >= 4
  7 ORDER BY Average_Rating DESC;
```

PRODUCT_ID	AVERAGE_RATING
104	5
101	4.5
103	4.5
102	4
105	4

12. Product Rating Report

```
SQL> SELECT
  2 Product_ID,
  3 COUNT(Rating_ID) AS Total_Ratings,
  4 AVG(Rating) AS Average_Rating,
  5 MAX(Rating) AS Highest_Rating,
  6 MIN(Rating) AS Lowest_Rating
  7 FROM Rating
  8 GROUP BY Product_ID
  9 ORDER BY Average_Rating DESC;
```

PRODUCT_ID	TOTAL_RATINGS	AVERAGE_RATING	HIGHEST_RATING	LOWEST_RATING
104	1	5	5	5
101	1	4.5	4.5	4.5
103	1	4.5	4.5	4.5
102	1	4	4	4
105	1	4	4	4